



n-gram language models

LING 430 Computational Linguistics Fall 2026

lecture 6b

Oct 1 2026

Siyu Liang

siyu.liang@rice.edu



last time

- probability
- conditional probability
- log probability



agenda

- language models
- n-grams
- training
- zeros & smoothing
- evaluation

today's notebook

class06b.ipynb

- course website → week 6 → activity
- **File → Save a copy in Drive**



language models

language model

- *The water of Walden Pond is so beautifully ____*
 - you can finish that: *blue, clear, green*
 - not *refrigerator*, not *this*
- a **language model** assigns a probability to each possible next word, and so to whole sentences
 - *all of a sudden I notice three guys standing on the sidewalk*: high
 - *on guys all I of notice sidewalk three a sudden standing the*: tiny
- working with language models is basically about how to get the probabilities

language model

why predict the next word?

- spelling and grammar
 - *Their are two midterms* is spelled correctly word by word, but the improbability of *Their are* can be used to disambiguate
- speech recognition
 - *It's easy to recognize speech* and *It's easy to wreck a nice beach* sound nearly the same, but one is more probable than the other
- assistive communication
 - a menu of likely next words for someone who selects words by eye gaze
- large language models are trained on exactly this task
 - their capability comes from predicting the next word billions of times

language model

why study the small one?

- n-gram models are the fruit fly of NLP: simple enough to build by hand, and every idea that matters for large models appears in them first
 - training and test sets
 - perplexity, the standard score
 - sampling to generate text
 - smoothing, backoff, interpolation
- still used where a large model is too heavy: predictive keyboards, spell checkers, systems on small devices

activity 1

- write down your top three guesses for the next word, best first
 - in *Emma*, the word after so is usually ____
 - the word after very is usually ____
 - the word after / is usually ____
- for each, how confident are you in your first guess: 90%? 50%? 10%?
- then run the notebook's first activity cell and compare with the novel

activity 1

debrief

- what you wrote down is a probability distribution over next words, with a context on the left of the bar: $P(\text{next word} \mid \text{so})$
- you are a language model, trained on every sentence you have read; the machine version does the same with a corpus and a count table
- in *Emma*
 - after *so*: *much* 96, *very* 79, *well* 30
 - after *very*: *much*, *well*, *good*
 - after *I*: *am*, *have*, *do*
- you probably matched on *I* and missed on *so*: your training data is not Austen's



n-grams

n-gram

- an **n-gram** is a window of n consecutive words, over *colorless green ideas sleep furiously*
 - unigrams: *colorless · green · ideas · sleep · furiously*
 - bigrams: (*colorless, green*) (*green, ideas*) (*ideas, sleep*) (*sleep, furiously*)
 - trigrams: (*colorless, green, ideas*) (*green, ideas, sleep*) (*ideas, sleep, furiously*)
- the same word, *n-gram*, also names the model that predicts a word from the $n - 1$ words before it

n-gram in Python

- `nltk.bigrams()` and `nltk.ngrams()` slide the window for you
- the windows can be characters instead of words
 - *unnatural: un, nn, na, at, tu, ur, ra, al*

```
words = "colorless green ideas sleep furiously".split()
```

```
list(nltk.bigrams(words))[:2]
```

```
→ [('colorless', 'green'), ('green', 'ideas')]
```

```
list(nltk.ngrams(words, 3))[0]
```

```
→ ('colorless', 'green', 'ideas')
```

n-gram

the chain rule

- a whole sentence, one word at a time, each given everything before it
 - $P(\textit{The water of Walden}) = P(\textit{The}) \times P(\textit{water} \mid \textit{The}) \times P(\textit{of} \mid \textit{The water}) \times P(\textit{Walden} \mid \textit{The water of})$
- true, and useless as it stands
 - the last factor needs a count of *Walden* after *The water of*; most histories occur once, or never
- language is creative: new sentences all the time, and no corpus holds enough copies of a long history to count

n-gram

the Markov assumption

- the **Markov assumption**: approximate the whole history by the last few words
 - instead of $P(\textit{blue} \mid \textit{The water of Walden Pond is so beautifully})$, settle for $P(\textit{blue} \mid \textit{beautifully})$
 - a bigram model conditions on one previous word, a trigram model on two
- false for language but useful anyway
- what it buys: histories short enough to count
- what it costs: everything long-distance, e.g. agreement across a clause, a pronoun and its antecedent
- the window is set by the data: a million words, trigrams; ten thousand, bigrams; the web, 5-grams



training

training

count and normalize

- $P(w \mid \text{previous}) = C(\text{previous } w) / C(\text{previous})$
 - Tuesday's conditional probability, and the maximum likelihood estimate
- each sentence gets a start marker `<s>` and an end marker `</s>`
 - “being the first word” and “ending here” become ordinary bigrams
- a training corpus of three sentences
 - `<s> I am Sam </s>`
 - `<s> Sam I am </s>`
 - `<s> I do not like green eggs and ham </s>`

activity 2

- by hand, from the three sentences: count and normalize, nothing else
 - $P(I \mid \langle s \rangle)$, $P(Sam \mid \langle s \rangle)$, $P(am \mid I)$
 - $P(do \mid I)$, $P(\langle /s \rangle \mid Sam)$, $P(Sam \mid am)$
- tips
 - the denominator is how often the first word occurs anywhere in the corpus
 - the markers count as words
- then run the notebook cell that trains on the same three sentences

activity 2

debrief

- that table is a complete, trained language model
- “trained on a corpus” means this, at any scale

estimate	count	value
$P(I \mid \langle s \rangle)$	2 of 3	0.67
$P(\text{Sam} \mid \langle s \rangle)$	1 of 3	0.33
$P(\text{am} \mid I)$	2 of 3	0.67
$P(\text{do} \mid I)$	1 of 3	0.33
$P(\langle /s \rangle \mid \text{Sam})$	1 of 2	0.50
$P(\text{Sam} \mid \text{am})$	1 of 2	0.50

training

in Python

- the same recipe on *Emma*: lowercase, markers on every sentence, two FreqDists, and a function for the division

```
train = []
for s in gutenbergsents("austen-emma.txt"):
...     low = [w.lower() for w in s]
...     train = train + ["<s>"] + low + ["</s>"]
unigrams = nltk.FreqDist(train)
bigrams = nltk.FreqDist(nltk.bigrams(train))
def p(w, prev):
...     return bigrams[(prev, w)] / unigrams[prev]
p("was", "she")
→ 0.1402
```

training

a real corpus

- bigram counts from the Berkeley Restaurant Project, 9,332 spoken queries (Jurafsky et al. 1994): row word first, column word next
- normalize each row by the row word's count: *i* 2,533, *want* 927, *to* 2,417, *eat* 746, *chinese* 158
 - $P(\textit{want} \mid \textit{i}) = 827 / 2,533 = 0.33$; $P(\textit{to} \mid \textit{want}) = 0.66$; $P(\textit{food} \mid \textit{chinese}) = 0.52$

	i	want	to	eat	chinese	food
i	5	827	0	9	0	0
want	2	0	608	1	6	6
to	2	0	4	686	2	0
eat	0	0	2	0	16	2
chinese	1	0	0	0	0	82

training

what the numbers know

- $P(\textit{to} \mid \textit{want}) = 0.66$: syntax, *want* takes an infinitive
- $P(\textit{to} \mid \textit{i}) = 0$: syntax too, no verb between them
- $P(\textit{chinese} \mid \textit{eat})$ above $P(\textit{english} \mid \textit{eat})$: not grammar, what people in Berkeley wanted to eat
- many sentences starting with *i*: a fact about the task, people talking to an assistant talk about themselves
- the model stores all of these without knowing which is which; the linguist reading the table can tell

training

the probability of a sentence

- chain the bigrams: $P(i \text{ want english food}) = P(i \mid \langle s \rangle) \times P(\text{want} \mid i) \times P(\text{english} \mid \text{want}) \times P(\text{food} \mid \text{english}) \times P(\langle /s \rangle \mid \text{food}) = 0.000031$
- small, and comparable: *i want chinese food* scores higher, since $P(\text{chinese} \mid \text{want})$ beats $P(\text{english} \mid \text{want})$
- in *Emma's* model, *she was not afraid .* chains to 3.8×10^{-8} , which is -7.4 on the log scale

```
sentence = "<s> she was not afraid . </s>".split()
probs = [p(w, prev) for prev, w in nltk.bigrams(sentence)]
[round(x, 4) for x in probs]
→ [0.0614, 0.1402, 0.0684, 0.0019, 0.0462, 0.7423]
sum(log10(x) for x in probs)
→ -7.4237
```

training

word order

- a unigram model multiplies word frequencies and ignores order
 - *she was not afraid* and *afraid not was she* score the same
- the bigram model scores the second one at zero: no *afraid not*, no *not was* in the training data
- one word of memory is enough to know that *she was* is a statement and *was she* a question



zeros & smoothing

zeros

- *Emma* has 7,341 word types, so 53.9 million possible bigrams
 - 64,263 of them occur about one in a thousand
- most missing ones are ungrammatical; many are ordinary English Austen did not happen to write
 - *ruby slippers* can be absent from a corpus that has both words
- a zero bigram gives the whole sentence probability zero, however good the rest of it is
- **zeros** are the central problem of n-gram modeling; the maximum likelihood estimate is never used as is

smoothing

add-one

- **Laplace smoothing** adds one to every count before normalizing
 - $P(w \mid \text{prev}) = (C(\text{prev } w) + 1) / (C(\text{prev}) + V)$
 - V is the vocabulary size: every possible next word got one extra count
- the mass has to come from somewhere
 - $P(\text{to} \mid \text{want})$ drops from 0.66 to 0.26 in the restaurant corpus: 1,446 zeros each took a share
- add-k adds a fraction instead of 1 and moves less; k is tuned on held-out data
- backoff and interpolation use the shorter n-gram when the longer one is missing: no *ruby slippers*, fall back on $P(\text{slippers})$



evaluation

evaluation

training and test sets

- a model is scored on text it did not see during training
 - **training set:** supplies the counts
 - **test set:** held out, touched once, at the end
 - **development set:** where choices like k are tuned, so the test set stays clean
- testing on the training data inflates every probability: data contamination
- the test set should look like the intended use, e.g. restaurant queries for a restaurant assistant

evaluation

perplexity

- the better model assigns the higher probability to the test set
- a probability shrinks with length, so it is normalized per word
 - **perplexity**: the inverse probability of the test set, to the power $1/N$
 - lower is better
- a weighted branching factor: perplexity 100 means the model is as uncertain at each step as a choice among 100 equally likely words
- 38 million words of the *Wall Street Journal*, tested on 1.5 million more:
unigram 962, bigram 170, trigram 109

evaluation

Emma against Persuasion

- train on *Emma*, test on *Persuasion*: the bigram model is half as perplexed
- 4.6% of the test tokens never appear in *Emma*; they are mapped to an unknown-word symbol first
- with add-one instead, the bigram model does worse than the unigram one, 904 against 549
 - too much probability moved to the zeros; smoothing is a choice, and the development set is where it gets made

model, add-k with $k = 0.01$	perplexity on Persuasion
unigram	680
bigram	352



next time

- Tuesday, Oct 6, in person
 - generation, perplexity, and sampling
- **HW 3 due Thursday night, 11:59 pm**